Project - High Level Design

On

CI/CD for Node.js Retail App

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	ANGEL GURJAR	EN22CS301123
2.	SAHIL SHRICHAND TALREJA	EN22CS301838
3.	KIRTI CHOUDHARY	EN22CS301508
4.	PAVANI KANOONGO	EN22CS301699
5.	MAHI VISHWAKARMA	EN22CS301568
6.	SAIYAD ISHAN ALI	EN22CS301839

Group Name: Group-01

Project Number: DO-01

Industry Mentor Name:

University Mentor Name:

Academic Year: 2022-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1. Scope of the Document

This document details the automation architecture from the moment a developer pushes code to a GitHub repository until a production-ready Docker image is stored in a private registry. It specifically addresses the integration of **GitHub Actions** and **Docker**.

1.2. Intended Audience

- **Lead Architects:** To verify compliance with enterprise delivery standards.
- **Security Teams:** To audit the vulnerability scanning and secrets management gates.
- **Site Reliability Engineers (SREs):** To understand the deployment artifacts and rollback triggers.

1.3. System Overview

The pipeline acts as the "Engine Room" for the Retail App. It transitions the application from raw JavaScript/TypeScript code into a secure, immutable container. It enforces a **"No-Test, No-Merge"** policy to maintain the stability of the retail platform.

2. System Design

2.1. Application Design

The Retail App is built using **Node.js (LTS version)**.

- **Framework:** Express.js or NestJS.
- **Packaging:** Dockerized using a multi-stage build to reduce the final image size (e.g., using node:18-alpine).

2.2. Process Flow

The pipeline is divided into three distinct logical "Stages":

1. **Commit Stage (The Gatekeeper):** Triggers on every push. Executes npm install, npm run lint, and npm test.
2. **Security Stage (The Auditor):** Runs npm audit and Snyk/Trivy scans to check for known vulnerabilities in dependencies.

3. **Release Stage (The Factory):** Triggers only on a merge to the main branch. It builds the Docker image, tags it with a unique semantic version (e.g., v1.2.3), and pushes it to the **GitHub Container Registry (GHCR)**.

2.3. Information Flow

- **Trigger Data:** Event payloads from GitHub Webhooks (Commit hash, author, branch).
- **Logs:** Real-time stdout/stderr from the GitHub Runner environment.
- **Status Updates:** Feedback sent back to the Pull Request UI (Green checkmark or Red 'X').

2.4. Components Design

- **Runner:** GitHub-hosted Ubuntu runners (2vCPU, 7GB RAM).
- **Secret Store:** GitHub Actions Secrets (Encrypted at rest).
- **Container Registry:** GHCR for private, enterprise-grade storage.

2.5. Key Design Considerations

- **Concurrency:** Use concurrency groups in GitHub Actions to cancel older, redundant builds if a new push is made to the same branch.
- **Immutable Artifacts:** Once a Docker image is built for production, it is never overwritten; it is tagged uniquely to allow for instant rollbacks.

2.6. API Catalogue

The pipeline leverages the **GitHub Check Runs API** to communicate the success or failure of tests back to the developer's PR interface.

3. Data Design

3.1. Data Model

- **Pipeline Artifacts:** .zip files of test results and coverage reports (lcov).
- **Container Manifests:** JSON-based manifests describing image layers and architecture (amd64/arm64).

3.2. Data Access Mechanism

- **OIDC (OpenID Connect):** Used for "Keyless" authentication between GitHub Actions and cloud providers (AWS/GCP/Azure) if the registry is hosted there.

3.3. Data Retention Policies

- **Artifacts:** Deleted after 14 days to save storage costs.
- **Release Images:** Retained indefinitely for compliance and historical auditing.

3.4. Data Migration

The CI/CD pipeline includes a **"Migration Verification"** step. Before deployment, it runs a "dry-run" of database migrations against a temporary containerized database to ensure schema changes won't break the production DB.

4. Interfaces

- **Developer Interface:** GitHub Pull Request page (Status checks).
- **Monitoring Interface:** GitHub Actions "Insights" tab to track build duration trends and failure rates.

5. State and Session Management

The pipeline is **Stateless**. However, to manage "State" across different jobs (e.g., passing a build folder from the 'Build' job to the 'Test' job), it uses:

- **Upload-Artifact:** Persists files within a single workflow run.
- **Download-Artifact:** Retrieves those files in subsequent jobs.

6. Caching

- **Dependency Cache:** Uses actions/cache for the ~/.npm directory. This reduces build time by **40–60%** by avoiding redundant downloads from the npm registry.

7. Non-Functional Requirements

7.1. Security Aspects

- **Branch Protection:** Rules are set so that the main branch cannot be pushed to directly; it *must* pass the CI pipeline via PR.
- **Least Privilege:** The GitHub Token (GITHUB_TOKEN) is scoped to read for contents and write only for packages.

7.2. Performance Aspects

- **Tree Shaking:** Node.js production dependencies are separated from devDependencies during the Docker build to ensure the image remains under **200MB**.

8. References

- *Node.js Best Practices (Security/Performance)*
- *GitHub Actions Documentation for Enterprise*
- *Docker Multi-stage Build Guide*